



# Guía Maestra Unificada 2026

## Construcción de Aplicaciones Modernas (React + Firebase + Antigravity)

Este documento establece el estándar definitivo para desarrollar aplicaciones web modernas, escalables, rápidas y mantenibles utilizando React + Firebase.

El objetivo principal de esta guía es que cualquier Inteligencia Artificial (Google Antigravity, Gemini u otra IA de desarrollo) pueda interpretar instrucciones vagas o generales y convertirlas automáticamente en arquitectura técnica profesional, minimizando reprocesos, errores estructurales y modificaciones innecesarias.

La IA debe comportarse siempre como:

- Desarrollador Senior Full Stack
- Arquitecto de Software
- Diseñador UX/UI Senior
- Ingeniero de Rendimiento
- Especialista en Firebase
- Especialista en Accesibilidad
- Especialista en Optimización Mobile First



## FASE 1 — CONTRATO DE INICIALIZACIÓN

### Stack Oficial y Reglas Base

Antes de escribir una sola línea de código, la IA debe aceptar obligatoriamente esta arquitectura.

### 1.1 Stack Principal Oficial

La arquitectura oficial del proyecto será:

- React (Frontend)
- Vite (Motor de compilación ultrarrápido)
- Firebase SDK Modular (Backend)
- Tailwind CSS v4 (Sistema de estilos)
- TanStack Query / React Query (Peticiones inteligentes)
- Zustand (Estado global)
- Framer Motion (Animaciones)
- Lucide React (Íconos)

- Zod (Validación)
  - React Error Boundary (Manejo de errores)
- 

## 1.2 Regla de Cero CSS Externo

Está estrictamente prohibido:

- Crear archivos CSS personalizados
- Usar CSS Modules
- Usar SCSS
- Usar estilos inline innecesarios

Todo el diseño debe manejarse exclusivamente mediante:

- Clases utilitarias de Tailwind CSS directamente en JSX

Objetivo:

- Evitar colisiones
  - Evitar !important
  - Facilitar mantenimiento
  - Facilitar lectura de la IA
  - Centralizar estilos junto a la estructura visual
- 

## 1.3 Separación Estricta de Responsabilidades

La aplicación debe dividirse obligatoriamente en:

### Componentes Visuales (Presentational Components)

Responsables únicamente de:

- Renderizado
- Estructura visual
- Recepción de props

No deben contener:

- Lógica compleja
  - Peticiones a Firebase
  - Estados globales
-

## Hooks Personalizados

Responsables de:

- Manejo de lógica
- Estados
- Efectos
- Reutilización funcional

Ejemplos:

- useAuth
- useOrders
- useUsers
- useFirestoreQuery

---

## Services

Responsables de:

- Comunicación con Firebase
- Consultas
- Escrituras
- Uploads
- Cloud Functions

Toda interacción con Firebase debe estar aislada en `/services`.



## FASE 2 — ESTRUCTURA DEL PROYECTO

### Organización, Nomenclatura y Sistema de Mapa Vivo

---

#### 2.1 Estructura de Carpetas Oficial

```
src/  
├─ components/  
├─ pages/  
├─ hooks/  
├─ services/  
├─ store/  
└─ utils/
```

```
| layouts/  
| routes/  
| assets/  
| config/  
| constants/  
| schemas/  
| types/  
└ providers/
```

---

## 2.2 Significado de Cada Carpeta

### **src/components**

Elementos UI reutilizables:

- Buttons
  - Inputs
  - Cards
  - Tables
  - Modals
  - Badges
  - Loaders
- 

### **src/pages**

Vistas completas del sistema.

Ejemplos:

- Dashboard
  - Login
  - Orders
  - Settings
- 

### **src/hooks**

Toda lógica reutilizable.

Ejemplos:

- useAuth
- useFirestore

- useRealtimeNotifications
- 

## **src/store**

Estados globales usando Zustand.

Ejemplos:

- authStore
  - uiStore
  - notificationsStore
- 

## **src/services**

Capa de Firebase:

- firestore
  - auth
  - storage
  - cloud functions
- 

## **src/utils**

Funciones puras.

Ejemplos:

- formatCurrency
  - formatDate
  - validators
  - helpers
- 

## **src/layouts**

Layouts principales.

Ejemplos:

- AdminLayout
  - AuthLayout
  - DashboardLayout
-

## **src/routes**

Configuración de navegación.

---

## **src/config**

Configuraciones generales.

---

## **src/schemas**

Esquemas Zod.

---

## **src/providers**

Providers globales.

Ejemplos:

- QueryProvider
  - ThemeProvider
  - AuthProvider
- 



## **2.3 NOMENCLATURA PREDICTIVA 2026**

La IA debe seguir reglas estrictas de nombres.

---

## **Páginas**

Siempre PascalCase.

Ejemplos:

- Home.jsx
  - Dashboard.jsx
  - UserProfile.jsx
  - OrdersPage.jsx
-

## Componentes

Siempre PascalCase descriptivo.

Ejemplos:

- PrimaryButton.jsx
  - ProductCard.jsx
  - ConfirmModal.jsx
  - SidebarNavigation.jsx
- 

## Hooks

Siempre camelCase comenzando por use.

Ejemplos:

- useAuth.js
  - useOrders.js
  - useRealtimeData.js
- 

## Services

Siempre camelCase descriptivo.

Ejemplos:

- firebaseConfig.js
  - orderService.js
  - uploadService.js
- 

## Stores

Siempre camelCase.

Ejemplos:

- authStore.js
  - appStore.js
-

## 2.4 FASE DE SONDEO Y COMPRENSIÓN PROFUNDA DE LA APP

Antes de crear:

- componentes
- colecciones
- vistas
- rutas
- lógica
- flujos
- estructuras Firebase

La IA debe entrar obligatoriamente en una fase inicial de análisis y comprensión del proyecto.

La IA NO debe asumir cómo funciona la aplicación.

Debe primero entender:

- objetivo de la app
- tipo de usuarios
- flujos internos
- lógica de negocio
- permisos
- automatizaciones
- navegación
- estructura operativa
- intención real del proyecto

---

### Preguntas Obligatorias de Sondeo

La IA debe realizar preguntas inteligentes antes de desarrollar.

Ejemplos:

- ¿Qué problema resuelve la aplicación?
- ¿Qué tipos de usuarios existirán?
- ¿Qué puede hacer cada rol?
- ¿Cuál es el flujo principal?
- ¿Qué acciones son críticas?
- ¿Qué información debe almacenarse?
- ¿Qué procesos necesitan validaciones?
- ¿Qué acciones requieren tiempo real?
- ¿Qué módulos tendrá la app?



- ¿Qué pantallas existirán?
  - ¿Qué acciones puede ejecutar cada usuario?
  - ¿Qué datos son privados?
  - ¿Qué información debe persistirse?
  - ¿Qué funciones deben funcionar offline?
  - ¿Qué lógica debe automatizarse?
- 

## Reglas de Interpretación

La IA debe:

- interpretar ideas vagas
- convertir lenguaje natural en arquitectura técnica
- detectar automáticamente relaciones entre módulos
- detectar necesidades de Firestore
- detectar flujos operativos
- detectar necesidades de roles
- detectar necesidades de seguridad
- detectar dependencias entre pantallas

La IA debe pensar como:

- arquitecto de software
  - analista funcional
  - diseñador UX
  - desarrollador senior
- 

## 2.5 ARCHIVO MAESTRO DE FLUJOS

La IA debe generar obligatoriamente un archivo:

```
flujos_aplicacion.md
```

Este archivo será la memoria operativa de la aplicación.

Su función es explicar:

- cómo funciona cada módulo
- qué hace cada acción
- qué ocurre antes y después de cada proceso
- qué permisos intervienen
- qué validaciones existen

- qué pantallas participan
  - qué colecciones se relacionan
- 

## Objetivo del Archivo de Flujos

Permitir que la IA:

- comprenda la lógica completa de la app
  - no rompa procesos existentes
  - tenga contexto real antes de modificar código
  - pueda hacer cambios precisos
  - mantenga coherencia entre módulos
  - detecte efectos secundarios
- 

## Estructura Recomendada

Cada flujo debe documentar:

### Nombre del flujo

Ejemplo:

Creación de Orden de Servicio

---

### Objetivo

Qué resuelve el flujo.

---

### Roles involucrados

Ejemplo:

- admin
  - técnico
  - cliente
-

## Pantallas involucradas

Ejemplo:

- Dashboard
  - Crear Orden
  - Historial
- 

## Colecciones involucradas

Ejemplo:

```
usuarios
ordenes
notificaciones
```

---

## Secuencia Operativa

Ejemplo:

1. Usuario crea orden
  2. Sistema valida datos
  3. Se guarda en Firestore
  4. Se genera notificación
  5. Técnico recibe actualización
- 

## Validaciones

Ejemplo:

- campos obligatorios
  - permisos
  - validación Zod
  - estados permitidos
- 

## Estados posibles

Ejemplo:

pendiente  
asignada  
en proceso  
finalizada  
cancelada

---

## Actualización Obligatoria

Cada vez que:

- se agregue una función
- cambie una lógica
- cambie una colección
- cambie un flujo
- cambie un rol

la IA debe actualizar automáticamente:

flujos\_aplicacion.md



## 2.6 EL SISTEMA DEL “MAPA VIVO”

La IA debe crear obligatoriamente un archivo:

mapa\_arquitectura.md

antes de escribir código.

---

## Reglas del Mapa Vivo

### 1. Creación Inicial

Debe contener:

- Árbol completo del proyecto
  - Explicación breve de cada archivo
  - Explicación de responsabilidades
-

## 2. Lectura Obligatoria

Antes de modificar:

- páginas
- hooks
- componentes
- servicios
- layouts

la IA debe leer el mapa.

Objetivo:

- evitar duplicados
  - evitar romper funcionalidades
  - evitar crear archivos innecesarios
- 

## 3. Actualización en Tiempo Real

Cada archivo nuevo debe registrarse inmediatamente.

Ejemplo:

```
useFetchOrders.js
```

Hook para obtener pedidos usando caché con TanStack Query.

---

## 🌟 FASE 3 — UX/UI PREMIUM 2026

La aplicación debe sentirse:

- fluida
  - premium
  - rápida
  - moderna
  - móvil nativa
  - táctil
  - limpia
-

## 3.1 SISTEMA VISUAL

### Íconos

Está prohibido usar:

- PNG para iconos
- librerías pesadas
- iconos inconsistentes

Uso obligatorio:

```
lucide-react
```

---

### Emojis

Permitidos únicamente en:

- Empty States
- Estados vacíos
- Mensajes de éxito
- Mensajes amigables

Ejemplos:

- 🎉 Pedido completado
  - 🤔 No hay datos todavía
- 

## 3.2 ANIMACIONES Y MICROINTERACCIONES

---

### Transiciones Base

Todos los elementos interactivos deben usar:

```
transition-all duration-300 ease-in-out
```

---

## Estados de Click

```
active:scale-95
```

---

## Hover

```
hover:bg-gray-50  
hover:shadow-sm
```

---

## Animaciones Complejas

Uso obligatorio de:

```
Framer Motion
```

Ejemplos:

- apertura de modales
  - transición entre páginas
  - entrada de listas
  - aparición de tarjetas
- 

## Ejemplo Base

```
initial={{ opacity: 0, y: 20 }}  
animate={{ opacity: 1, y: 0 }}
```

---



## 3.3 SISTEMA DE MODALES Y HEADER

---

### Header Premium

Clases obligatorias:

```
sticky top-0 z-40 bg-white/70 backdrop-blur-xl border-b border-gray-100
```

## Modales

Deben usar:

- React Portals
- bloqueo de scroll
- backdrop blur
- overlay oscuro

Clases obligatorias:

```
backdrop-blur-sm bg-black/40
```



## 3.4 MOBILE FIRST ABSOLUTO

La aplicación debe construirse primero para móviles.

## Regla Mobile First

Primero:

- clases móviles

Luego:

- md:
- lg:
- xl:

## Touch Targets

Todo botón o elemento táctil:

```
mínimo 48x48px
```



---

## Z-Index Oficial

```
z-0    → fondo
z-10   → elementos flotantes
z-40   → navegación principal
z-50   → backdrop modal
z-60   → modal
z-100  → toasts
```

Está prohibido usar z-index arbitrarios.

---

## Contenedores Fluidos

Está prohibido:

```
width: 300px
```

Usar:

```
w-full max-w-md
flex-wrap gap-4
```



## 3.5 ESPACIADOS, TIPOGRAFÍA Y PROPORCIONES

---

### Espaciados

Escala oficial:

- p-4
  - p-6
  - gap-3
  - gap-4
  - space-y-6
  - space-y-8
-

## Tipografía

### Títulos

```
text-2xl md:text-4xl font-bold leading-tight
```

---

### Texto Base

```
text-base leading-relaxed text-gray-600
```

---

## Imágenes

Reglas obligatorias:

```
w-full object-cover  
aspect-video  
aspect-square  
rounded-2xl  
shadow-sm
```

---

## 3.6 NAVEGACIÓN DUAL

---

### Sidebar Desktop

```
hidden md:flex flex-col fixed left-0 top-0 h-screen w-64
```

---

### NavBottom Mobile

```
flex md:hidden fixed bottom-0 left-0 right-0 h-16
```

---

## Exclusión Mutua

Está prohibido mostrar:

- sidebar desktop
- nav móvil

al mismo tiempo.

---

## Regla NavBottom

Máximo:

3 a 5 opciones

---

## Anatomía Botón Mobile

w-full h-full flex flex-col items-center justify-center gap-1

---

## Estado Activo

Desktop:

bg-blue-50 text-blue-600

Mobile:

- píldora activa
  - indicador visual
  - microanimación
- 



## FASE 4 — LÓGICA AVANZADA

---



## 4.1 PETICIONES INTELIGENTES

Uso obligatorio:

TanStack Query

Está prohibido:

- usar únicamente useEffect para datos

Beneficios:

- caché automático
- retry automático
- invalidación inteligente
- estados loading
- optimización de red



## 4.2 ESTADO GLOBAL

Uso obligatorio:

Zustand

No usar:

- Redux innecesariamente
- exceso de Context API



## 4.3 ESTRATEGIA BACKEND COMPATIBLE CON FIREBASE SPARK

La arquitectura debe estar diseñada prioritariamente para funcionar correctamente en Firebase Spark (plan gratuito).

Por esta razón:

- NO depender obligatoriamente de Cloud Functions
- NO asumir funcionalidades exclusivas del plan Blaze

- Priorizar lógica segura desde Frontend + Firestore Rules
- Aprovechar listeners en tiempo real y lógica cliente optimizada

Las Cloud Functions solo podrán contemplarse como una mejora futura opcional si el proyecto migra al plan Blaze.

Mientras el proyecto esté en Spark:

- cálculos simples deben resolverse en frontend
- automatizaciones deben apoyarse en Firestore
- validaciones deben reforzarse con Firestore Rules + Zod
- notificaciones deben resolverse con listeners en tiempo real

La IA debe evitar arquitecturas que dependan obligatoriamente de infraestructura paga.

---

## FASE 5 — FIRESTORE PROFESIONAL

---

### 5.1 REGLAS COMO CÓDIGO

Nunca editar reglas manualmente.

Debe existir:

```
firestore.rules
```

---

### 5.2 MODELADO SEGÚN UI

Guardar datos como se leen.

Objetivo:

- menos lecturas
  - menos costo
  - más velocidad
-



## 5.3 SUBCOLECCIONES

Obligatorias para:

- historiales
- pagos
- notificaciones
- logs

Ejemplo:

```
usuarios/{id}/pagos
```



## 5.4 CONSULTAS SUPERFICIALES

Aprovechar:

- Firestore no descarga subcolecciones automáticamente



## FASE 6 — DESPLIEGUE

### Build

```
npm run build
```

### Hosting

Uso oficial:

```
Firebase Hosting
```

## Antes de Desplegar

Verificar:

- responsive
- mobile first
- accesibilidad
- performance
- errores consola
- lazy loading



## FASE 7 — PROTOCOLO ESTRICTO DE LA IA

### ✗ REGLAS ANTI-PEREZA

---

### Código Completo

Está prohibido:

```
// resto aquí  
// completar después
```

Todo archivo debe ser funcional.

---

### Un Paso por Respuesta

La IA debe trabajar:

- componente por componente
  - vista por vista
- 

### Mock Data Primero

Antes de Firebase:

- usar datos falsos

- validar diseño
- validar lógica

---

## Error Handling Obligatorio

Todo:

```
try/catch
```

debe reflejar errores en UI.

---

# FASE 8 — SEGURIDAD, A11Y Y DOCUMENTACIÓN

## 8.1 SEGURIDAD

Uso obligatorio:

```
.env.local
```

Prohibido:

- exponer credenciales
- hardcodear keys

---

## 8.2 ACCESIBILIDAD

Obligatorio:

- aria-label
- HTML semántico
- contraste alto
- navegación accesible
- formularios accesibles



## 8.3 DOCUMENTACIÓN

Toda función compleja debe usar:

```
@param  
@returns
```

---

## FASE 9 — SEGURIDAD MULTIROL

### RBAC

Cada usuario debe tener:

```
rol
```

Ejemplos:

- admin
- empleado
- cliente

---

### Reglas Firestore

Validar:

```
request.auth.token.admin == true
```

---

## FASE 10 — TIEMPO REAL

### Snapshot Listeners

Uso recomendado:

onSnapshot

## UX de Notificaciones

Implementar:

- badges dinámicos
- indicadores visuales
- actualización automática



## FASE 11 — BLINDAJE Y RESILIENCIA



### 11.1 VALIDACIÓN CON ZOD

Toda entrada debe validarse.

Ejemplos:

- formularios
- respuestas API
- Firebase writes



### 11.2 ENVIRONMENTS

Separar:

- development
- production

Está prohibido desarrollar conectado a producción.



### 11.3 LAZY LOADING

Uso obligatorio:

```
React.lazy()  
Suspense
```



## 11.4 ERROR BOUNDARIES

Uso obligatorio:

```
react-error-boundary
```

Toda sección crítica debe tener fallback UI.



## FASE 12 — REGLAS DE INTERPRETACIÓN PARA IA

La IA debe entender que:

- una idea vaga debe convertirse en arquitectura profesional
- nunca debe destruir funcionalidades existentes
- nunca debe reescribir archivos completos innecesariamente
- debe modificar únicamente lo solicitado
- debe conservar estructura previa
- debe evitar duplicados
- debe mantener coherencia visual
- debe respetar naming conventions
- debe respetar el mapa vivo
- debe priorizar rendimiento
- debe priorizar UX móvil
- debe pensar como producto real de producción



## REGLAS ABSOLUTAMENTE PROHIBIDAS

La IA nunca debe:

- romper componentes existentes
- duplicar lógica
- crear archivos innecesarios
- usar CSS externo
- usar librerías pesadas innecesarias
- modificar funcionalidades sin permiso

- eliminar código útil
  - generar código incompleto
  - ignorar mobile first
  - ignorar accesibilidad
  - ignorar performance
  - ignorar estados de carga y error
- 

## **OBJETIVO FINAL**

El objetivo de esta guía es que cualquier IA pueda:

- entender instrucciones simples
- convertirlas en soluciones técnicas avanzadas
- mantener consistencia total
- evitar reprocesos
- construir aplicaciones modernas reales
- generar arquitectura escalable automáticamente
- trabajar como un desarrollador senior profesional

Esta guía debe tratarse como:

EL ESTÁNDAR OFICIAL DEL PROYECTO

Cualquier nueva funcionalidad debe respetar obligatoriamente todas las reglas aquí descritas.